

Advanced Programming

Project assignment – 4th stage

JavaFX

Images

Stage 4

- All students must update the project and *checkout* the `main` *branch*
- The student responsible for this task should start by creating a new *branch* from the `main` *branch*
 - *Branch* name: **stg4boardfx**
 - The student who creates the *branch* should *push* it to *GitHub* to make it available to the rest of the group
- The remaining students should update the project and then *checkout* the **stg4boardfx** *branch*

Stage 4

- The deadline to perform the *Pull Request* and the *Squash and Merge* is 23:59 on the day before the lesson during the week of May 5-9
 - Worker-students should complete this stage by 2025.05.10
- The *Squash and Merge* into the *main* branch must be performed on the *GitHub* website
 - The *branch* must not be deleted after the *Merge* is completed

Stage 4

- Create and configure a *JavaFX* component that should follow the internal structure of an MVC@PA component to represent the chess game board
 - **This component should extend the *JavaFX* Canvas**
- The new component should support:
 - Drawing the board
 - Drawing images for each chess piece
 - Moving a chess piece

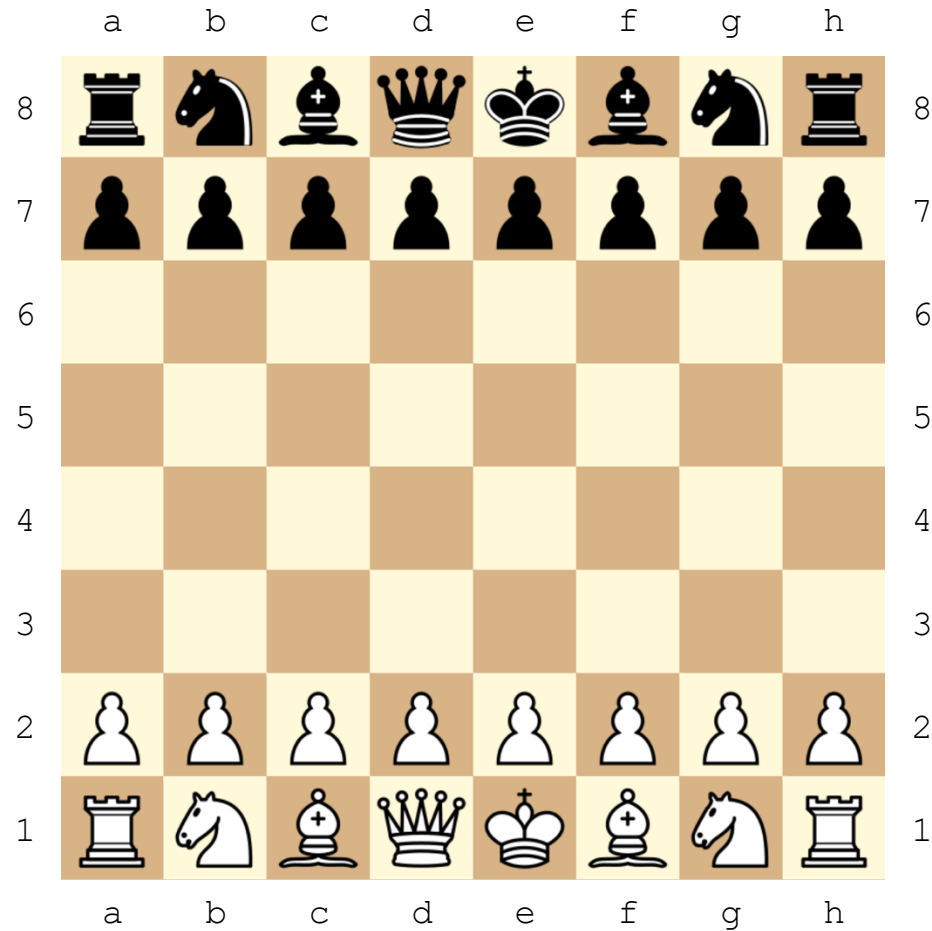
Drawing the board

- The board size should be dynamic
 - Although the size may vary, the board is always square
- The size should be retrieved from the data model
- The board should include labels for each column and row, with columns labeled from 'a' onward and rows numbered from 1 at the bottom to the maximum row number at the top, based on the board's size

Drawing pieces

- The piece type for each square should be retrieved from the data model
- Drawing images for each chess piece
 - Each group may use the images provided with the base project template or other images chosen by the group members
 - Images should be centered within each square

Chess board example



Moving a chess piece

- To move a piece, the current player should be able to...
 - Select the piece to move
 - Select the destination square
- Move validation should be handled by the model

Stage 4

- The created component must be included in the main window (`Stage`) of the application
- The UI should also display:
 - The names of the players
 - The current player (the one who makes the next move)
- When the game ends (either a winner is determined or it's a draw), the result should be shown, and the option to move a piece should be disabled
- Note:
 - The learning mode, including the *undo* and *redo* options, should be inactive at this stage
 - The implementation of these options will be defined in a later stage

Advanced Programming

JavaFX Resources
Images

Resources

- Applications may need extra information to...
 - Function properly, or
 - Enrich functionality and user interaction
- This information (usually constituted by auxiliary files) is called *Resources*

Resources

- This information can be of several types, for example:
 - Files with static information to start data structures or databases created by the application
 - Images to improve the representation of information
 - Sounds to provide feedback to user actions
 - Use of fonts best suited to the perception of a given textual information
 - among others...

Resources

- The easiest way to integrate these *Resources* into applications is to provide the files that represent them (the "real resources" themselves) within the application
 - However, the process of providing full paths to the files, to allow them to be opened, is not always simple

Resources

- Through the representative type of a class, obtained using `MyClass.class` or by calling `myObj.getClass()` from a reference to an object of that type, it is possible to have access to methods that facilitate the reading of resources
- For this operation to be feasible, the files must be placed in the same package of these classes or in sub-packages

Resources access

- To get a URL for a resource file
 - `MyClass.class.getResource(String name)`
 - Returns a URL for the resource, for example,
`file:///C:\xpto\otpx.txt`
 - The file path of a file represented by a URL can be obtained using the `getFile()` method
- To get an `InputStream` to the file
 - `MyClass.class.getResourceAsStream(String name)`
 - Returns an `InputStream` that can be used to read the respective information

Example of a utility class

```
public class ResourceLoader {  
    private ResourceLoader() { }  
  
    public static InputStream getResourceFile(String resource) {  
        return ResourceLoader.class.getResourceAsStream(resource);  
    }  
  
    public static String getResourceFilename(String resource) {  
        return ResourceLoader.class.getResource(resource).getFile();  
    }  
}
```

- package pt.isec.pa.xpto
 - package model
 - package ui
 - package resources
 - **class ResourceLoader**
 - package images
 - logo.png
 - diagram.jpg
 - package sounds
 - intro.mp3
 - music.mp3

Example of use:

```
String filename = ResourceLoader.getResourceFilename("images/logo.png");
```


Helper class for images (with cache)

```
public class ImageManager {
    private ImageManager() { }
    private static final HashMap<String, Image> images = new HashMap<>();
    public static Image getImage(String filename) {
        Image image = images.get(filename);
        if (image == null)
            try (InputStream is = ImageManager.class.getResourceAsStream("images/"+filename)) {
                image = new Image(is);
                images.put(filename, image);
            } catch (Exception e) { return null; }
        return image;
    }
    public static Image getExternalImage(String filename) {
        Image image = images.get(filename);
        if (image == null)
            try {
                image = new Image(filename);
                images.put(filename, image);
            } catch (Exception e) { return null; }
        return image;
    }
    public static void purgeImage(String filename) { images.remove(filename); }
}
```

Helper class for images (with cache)

- Example of package organization:

- package pt.isec.pa.xpto
 - package model
 - package ui
 - package resources
 - class ImageManager
 - package images
 - logo.png
 - diagram.jpg

- Examples of use:

```
GraphicsContext gc = canvas.getGraphicsContext2D();
gc.drawImage(ImageManager.getImage("logo.png"), 150, 150, 200, 100);
gc.drawImage(
    ImageManager.getExternalImage(
        "https://www.oracle.com/img/tech/cb88-java-logo-001.jpg"),
    250, 50, 50, 100);

ImageView imgv = new ImageView(ImageManager.getExternalImage(
    "https://www.oracle.com/img/tech/cb88-java-logo-001.jpg"));
imgv.setPreserveRatio(true);
imgv.setFitWidth(150);
vbox.getChildren().add(imgv);
```